

AI Study Companion

Candidate Challenge Documentation • Candidate: Dhana Gunda

AI Prompts Used During Development

AI Study Companion (Candidate Challenge Edition)

This document contains the actual prompts materially used with AI development tools (Antigravity Agentic Assistant / Gemini 2.5) during the lifecycle of the **AI Study Companion** project, organized by architectural layer and functional phase.

1. System Architecture & Project Scaffolding

Prompt 1.1: Core Topology & Fail-Loud PostgreSQL Design

"Design a clean modular hexagonal architecture for an AI Study Companion platform matching the PRD specification. Enforce a strict multi-tenant boundary hierarchy: Space > Project. Each Project must maintain its own learning context, uploaded materials, knowledge chunks, AI tutor sessions, adaptive quizzes, concept mastery, and growth analytics. Eliminate any silent fallbacks to SQLite in production paths: the system must strictly require PostgreSQL 16 with the pgvector extension and fail loudly with descriptive diagnostics if PostgreSQL or the vector extension is missing."

Prompt 1.2: Directory Structure & Separation of Concerns

"Establish a clean full-stack monorepo layout with Next.js 14 App Router on the frontend and FastAPI on the backend. Structure the backend into domain-driven modules: `knowLedge` (ingestion, PyMuPDF parsing, vector embeddings), `ai` (client abstraction, prompt composer, refusal guardrail), `assessment` (adaptive quiz selector, rubric evaluator), and `mastery` (EMA score updater, growth trend detector). Ensure Redis is integrated for asynchronous background job queuing."

2. Database Schema & Vector Indexing

Prompt 2.1: Domain Models with Strict Project Boundaries

"Implement SQLAlchemy 2.0 async models for all 16 core entities: `User`, `Space`, `Project`, `Material`, `DocumentChunk` (with `Vector(1536)`), `Concept`, `ConceptMastery`, `MasterySnapshot`, `Conversation`, `Message`, `QuizSession`, `Question`, `Answer`, `LearningContext`, `LearningEvent`, and `AIRequestLog`. Crucially, enforce a direct indexed foreign key `project_id` on every project-scoped table to guarantee zero-join query isolation and prevent cross-tenant data leakage."

Prompt 2.2: Comprehensive Demo Seed Generator

"Create an idempotent database seed script (`app/seed.py`) that initializes the Neon PostgreSQL database with a demo learner (`demo@studycompanion.ai`), an admin (`admin@studycompanion.ai`), a 'Computer Science' Space, an 'Intro to Machine Learning' Project, 4 extracted Concepts (Supervised Learning, Loss Functions, Gradient Descent, Overfitting & Regularization), 12 historical MasterySnapshots illustrating an active learning curve, and an initial Next Action recommendation."

3. Grounded AI Tutor & Guardrail Engineering

Prompt 3.1: Evidence Guardrail & Low-Evidence Refusal

"Write a context composer and guardrail service for the AI Tutor. Retrieve top-4 document chunks using cosine similarity from `document_chunks` . If the maximum similarity is below 0.60 or if the active project contains no ingested chunks, halt speculation and return a structured refusal: 'I do not have sufficient information in your uploaded project materials to answer this question reliably. Please upload relevant notes or a PDF covering this topic.' Log the refusal event to `ai_request_logs` with status `REFUSED_LOW_EVIDENCE` ."

Prompt 3.2: Streaming SSE Chat with Provenance Citations

"Implement a Server-Sent Events (SSE) chat endpoint `POST /api/v1/projects/{id}/tutor/chat` . Stream response tokens in real-time. In the system prompt, strictly instruct the LLM to ground its reasoning in the provided text chunks and cite the exact source using `[Source: {filename} - Page {page_number}]` . Ensure prompts are immune to injection by segregating untrusted user PDF text from system instruction boundaries."

4. Adaptive Assessment & Concept Mastery Engine

Prompt 4.1: Dynamic Weak-Concept Targeting

"Design an adaptive quiz generator that selects concepts requiring attention (`trend == 'ATTENTION'` or lowest `mastery_score`). Generate a mix of Multiple Choice Questions and Open-Ended conceptual questions with rubrics. Return a structured `QuizSession` object."

Prompt 4.2: Exponential Moving Average (EMA) Mastery Tracking

"Implement the concept mastery scoring algorithm using Exponential Moving Average: $M_t = \alpha \cdot s_t + (1 - \alpha) \cdot M_{t-1}$, where $\alpha = 0.3$. Bound the score between 0.0 and 1.0 . Dynamically calculate the growth trend: assign `IMPROVING` if the delta exceeds $+0.05$, `ATTENTION` if it drops below -0.05 or is under 0.50 , and `STABLE` otherwise. Persist historical snapshots in `mastery_snapshots` ."

5. Frontend & UI/UX Development

Prompt 5.1: Next.js 14 App Router & Live Connectivity Probe

"Build a responsive Next.js 14 dark-themed frontend using Tailwind CSS and shadcn/ui. On the landing page (/), create a real-time 'Live Stack Connectivity Probe' card that queries /api/v1/health and displays live connectivity badges for PostgreSQL 16 + pgvector, Redis Queue, and AI Provider status. Provide instant visual confirmation that the cloud stack is fully operational."

Prompt 5.2: Project Workspace with 5-Tab Learning Loop

"Develop the Project Workspace at /spaces/[spaceId]/projects/[projectId]. Implement 5 cohesive tabs representing the learning loop:

1. Overview: High-level mastery %, weak concepts, recent timeline, and active recommendation card.
2. Materials: PDF upload zone with async processing status table.
3. AI Tutor: Real-time SSE streaming dialogue with interactive citation badges.
4. Adaptive Quiz: Multi-question interactive quiz runner with instant grading.
5. Mastery: Concept mastery progress bars and EMA trend badges."

6. Debugging, Cloud Deployment & DevOps

Prompt 6.1: Vercel TypeScript Compilation Fix

"Fix TypeScript compilation error during npm run build on Vercel where Artifact and Message types were missing from frontend/src/Lib/api.ts. Export Artifact, Message, and enhance Citation to ensure backwards compatibility across all chat and viewer components."

Prompt 6.2: Resilient Cloud Connection & Asyncpg URL Normalization

"Resolve [Errno 111] Connection refused on Render. Modify backend/app/core/config.py to automatically normalize PostgreSQL URLs for asyncpg (converting postgresql:// to postgresql+asyncpg:// and stripping query parameters like channel_binding which asyncpg does not support). Enforce live Neon PostgreSQL connection in production, and configure graceful standalone in-memory fallback for Redis."

7. Automated Testing & Verification

Prompt 7.1: Pytest Test Suite across Business Domains

"Write a comprehensive automated pytest test suite in app/tests/:

1. test_auth_isolation.py: Password hashing, JWT token encoding/decoding, and multi-tenant project boundary rejection (HTTP 403).
2. test_grounding.py: Evidence threshold evaluation, low-evidence refusal triggering, and citation formatting.

3. `test_adaptive_quiz.py` : Adaptive concept selection prioritizing weak areas, EMA mastery score bounds, and recommendation generation.
4. `test_injection.py` : Prompt-injection defense boundary validation."